Solution: Number of Provinces

Problem Statement

There are `n` cities. Some of them are connected in a network. If City `A` is directly connected to City `B`, and City `B` is directly connected to City `C`, city `A` is directly connected to City `C`.

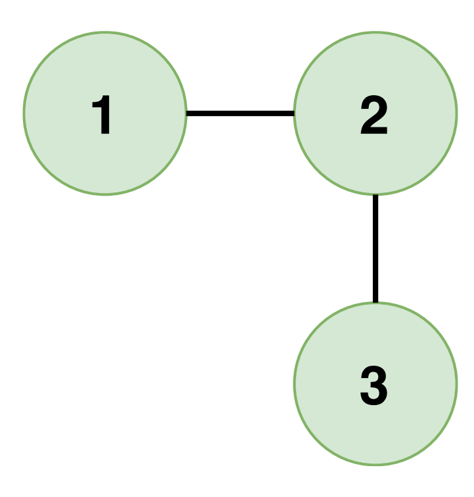
If a group of cities are connected directly or indirectly, they form a **province**.

You are given a square matrix of size `n x n`, where each cell's value indicates whether a direct connection between cities exists (1 for connected and 0 for not connected).

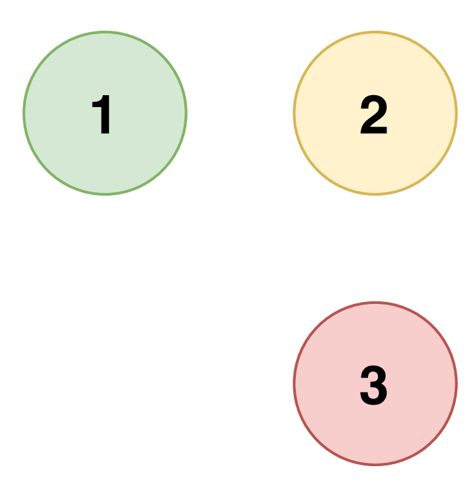
Determine the total number of provinces.

Examples

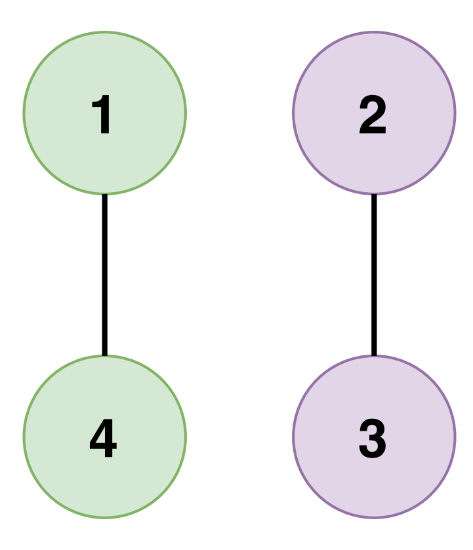
- **Example 1**:
  - Input: `[[1,1,0],[1,1,1],[0,1,1]]`



- Expected Output: `1`
- Justification: Here, all cities are connected either directly or indirectly, forming one province.
- **Example 2**:
  - Input: `[[1,0,0],[0,1,0],[0,0,1]]`



- Expected Output: `3`
- Justification: In this scenario, no cities are connected to each other, so each city forms its own province.
- **Example 3**:
  - Input: `[[1,0,0,1],[0,1,1,0],[0,1,1,0],[1,0,0,1]]`



- Expected Output: `2`
- Justification: Cities 1 and 4 form a province, and cities 2 and 3 form another province, resulting in a total of 2 provinces.

Solution

To solve this problem, we'll employ the Union-Find algorithm, a powerful tool for managing disjoint sets. The essence of the approach is to group cities into sets, where each set represents a connected province.

Initially, every city is considered its own province. As we go through the matrix, we use Union operations to join cities that are directly connected, effectively merging their sets. The Find operation identifies the representative of each set (or province) and helps in determining if two cities are already in the same province. By iteratively applying Union operations to all connected city pairs, we merge their respective sets.

In the end, the number of distinct sets (or provinces) is determined by counting the unique representatives of each set, providing the solution to our problem. This approach ensures efficient handling of connections and an accurate count of disconnected provinces.

Here's a detailed explanation:

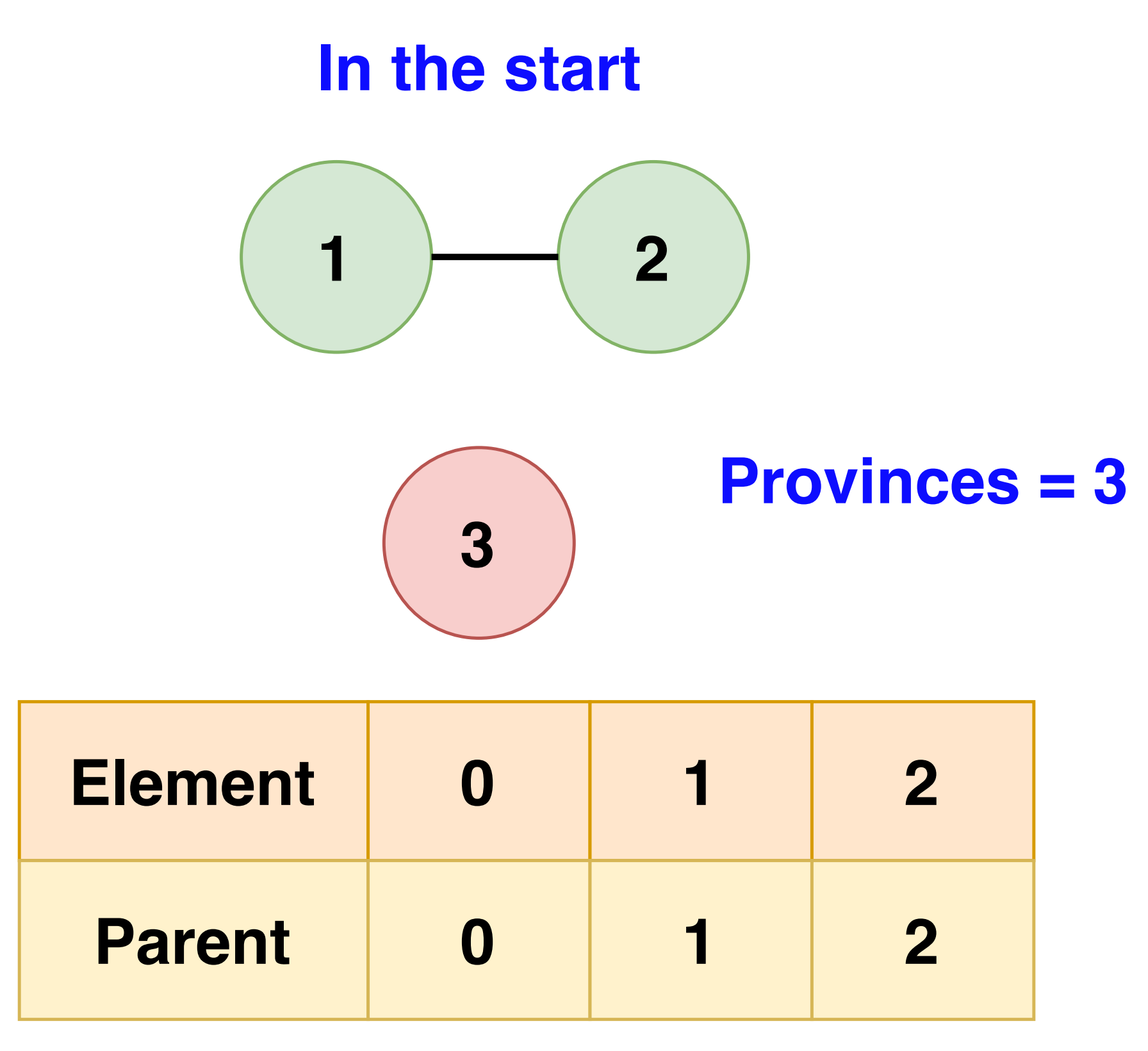
1. **Initialization**: We initialize an array `parent` to keep track of the root of each node. Each node is initially its own parent.
2. **Union Operation**: If two nodes belong to different sets (different parents), we connect them by making the root of one the parent of the other, effectively merging the sets.
3. **Find Operation with Path Compression**: When we wish to determine which set a node belongs to, we find its root. If during this search we traverse nodes, we directly connect them to the root to flatten the structure, which speeds up future 'find' operations.
4. **Counting Provinces**: We start with the assumption that each node is its own province. As we connect nodes, we decrement the count of provinces.

This approach is efficient due to path compression, which flattens the tree structure used internally, leading to very fast subsequent find operations.

Algorithm Walkthrough

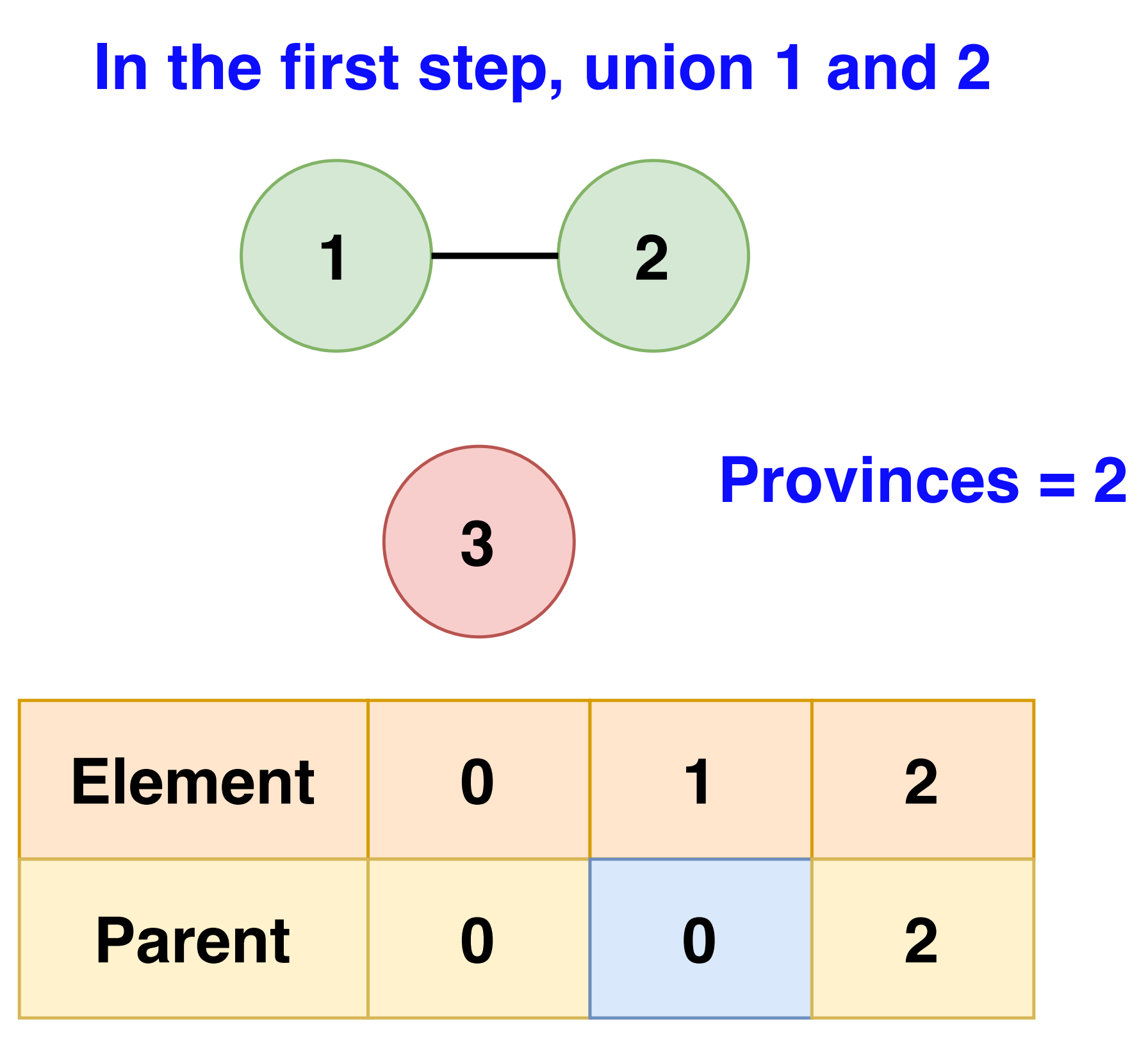
Given an input `isConnected = [[1,1,0],[1,1,0],[0,0,1]]`, let's walk through the algorithm:

- Initialize parent to [0, 1, 2].



Element	0	1	2
Parent	0	1	2

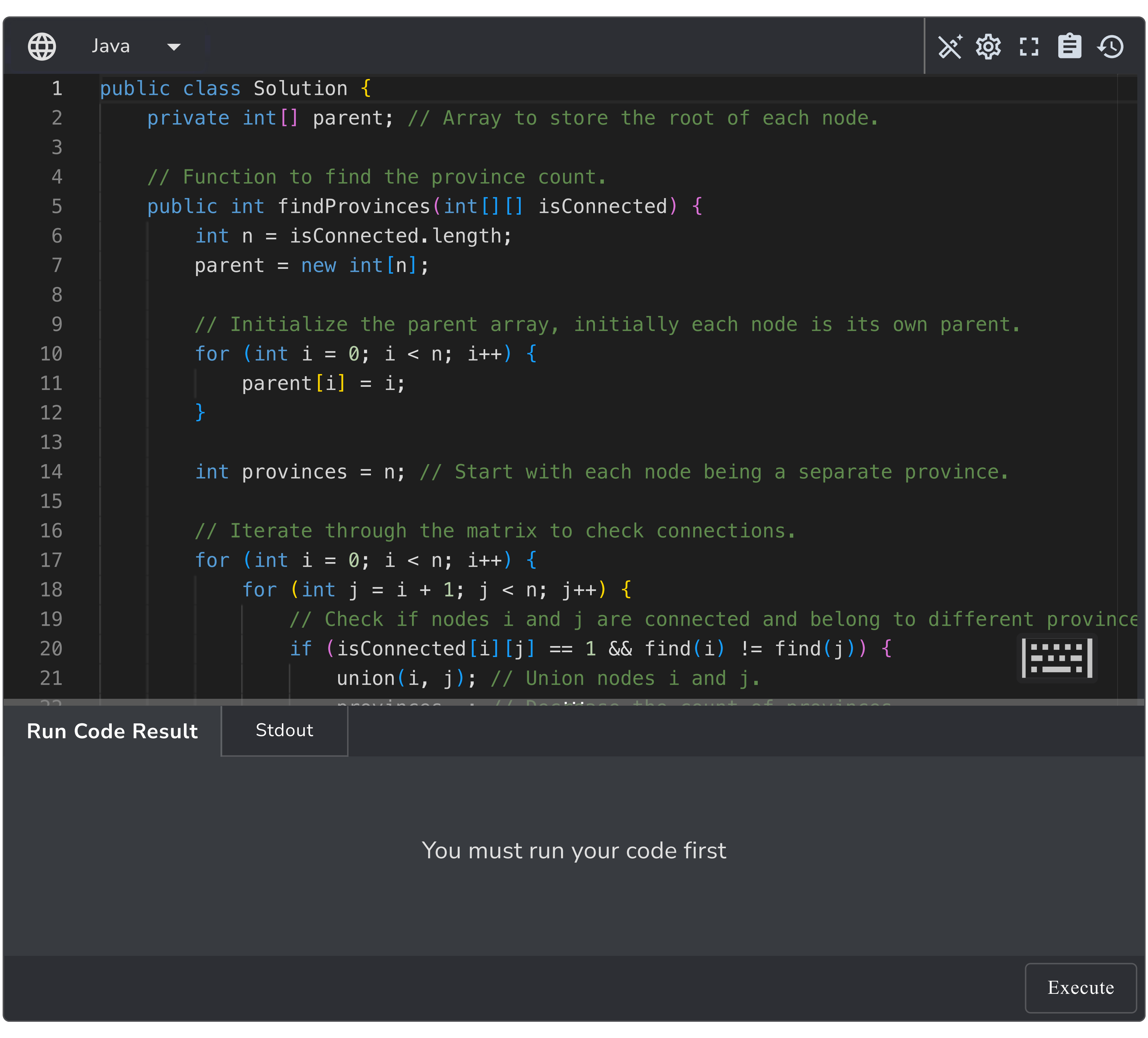
- Compare node 0 with node 1, they are connected. Union them, parent becomes [0, 0, 2]. Provinces decrement to 2.



Element	0	1	2
Parent	0	0	2

- Compare node 0 with node 2, no connection. Skip.
- Compare node 1 with node 2, no connection. Skip.
- The final count of provinces is 2.

Code



```
public class Solution {
    private int[] parent; // Array to store the root of each node.

    // Function to find the province count.
    public int findProvinces(int[][] isConnected) {
        int n = isConnected.length;
        parent = new int[n];

        // Initialize the parent array, initially each node is its own parent.
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }

        int provinces = n; // Start with each node being a separate province.

        // Iterate through the matrix to check connections.
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                // Check if nodes i and j are connected and belong to different province
                if (isConnected[i][j] == 1 && find(i) != find(j)) {
                    union(i, j); // Union nodes i and j.
```

You must run your code first

Complexity Analysis

Time Complexity: The time complexity is O(n^3) in the worst case, where n is the number of nodes. This is because the union and find operations can take O(n) time without optimizations and we perform up to O(n^2) comparisons.

However, with path compression and union by rank, the amortized time complexity of each union-find operation can be reduced to nearly O(1), making the algorithm run in nearly O(n^2) time.

Space complexity: The space complexity is O(n) for storing the parent array.

← Previous
Number Of Provinces (Medium)
Next →
Is Graph Bipartite? (Medium)
☐ Mark as Completed